

RUINPRINT

Documentação técnica de desenvolvimento

Editor desktop de imagens com estética de impressão, filme e desgaste analógico

VERSÃO	TECNOLOGIAS	PLATAFORMA DE DISTRIBUIÇÃO
1.0.0	Python, PySide6, Pillow, NumPy e OpenCV	Windows - ONEDIR portátil e instalador Inno Setup

Este documento descreve a implementação observada no código-fonte do RuinPrint, seu fluxo de processamento, persistência, exportação e empacotamento. Foi preparado para apoiar manutenção, revisão técnica e apresentação profissional do projeto.

REVISÃO DO DOCUMENTO: 14 de outubro de 2026

Capturas reais do aplicativo estão nas páginas 12-15; arquivos originais: docs/screenshots/.

1. Visão geral

RuinPrint é um aplicativo desktop em Python para transformar imagens em composições de tons limitados, inspiradas em impressos antigos, gravura, filme e reprodução degradada. O usuário importa uma ou mais imagens, ajusta controles visuais, acompanha uma prévia e exporta um PNG em resolução completa.

Objetivo e escopo

O projeto privilegia uma cadeia de processamento explícita e configurável. Os módulos de interface coletam parâmetros; um snapshot imutavel de PipelineSettings alimenta tanto a prévia quanto a exportação. O estado editável e mantido separado dos pixels originais para permitir recalculo não destrutivo.

Estado observado

- Versão declarada centralmente: 1.0.0.

- Interface PySide6 em português brasileiro, com comparação lado a lado, várias imagens, presets, desfazer/refazer e exportação PNG.

- Build Windows configurada como PyInstaller ONEDIR, instalador per-user via Inno Setup e pacote portátil.

- A pasta de trabalho contém alterações locais ainda não commitadas. Esta documentação descreve a árvore atual; a revisão dessas alterações deve preceder um release público.

Tecnologias

TECNOLOGIA	USO NO PROJETO
Python	Aplicação, orquestracao e ferramentas de build.
PySide6 / Qt	Janela desktop, controles, sinais, workers e persistência de preferências.
Pillow	Leitura e escrita de imagem, orientação EXIF, RGBA, miniaturas e metadata PNG.
NumPy	Operações vetorizadas em arrays de pixels.
OpenCV	Conversões de cor, cálculos de paleta e processamento de imagem.
PyInstaller / Inno Setup	Aplicação Windows ONEDIR e instalador.

2. Arquitetura

A aplicação separa apresentação, estado do documento, configuração visual, efeitos e empacotamento. O ponto de entrada cria QApplication e MainWindow. A janela coordena a sessão e delega leitura, pipeline, presets e exportação a módulos de core e workers.

CAMADA	RESPONSABILIDADE	MÓDULOS PRINCIPAIS
Entrada	Inicializar Qt e apresentar erros de pasta de dados.	app.py
Interface	Painel de controles, lista de imagens, vistas sincronizadas e ações.	ui/main_window.py, controls.py, image_view.py, theme.py
Documentos e estado	Imagem original, prévia e snapshot opcional recuperado do PNG.	core/image_document.py, image_loader.py
Configuração e regras	Dataclasses, validação/migração, preset, histórico e caminhos.	core/pipeline.py, processing_settings.py, preset_manager.py, edit_history.py, app_paths.py
Efeitos	Transformações locais de tom, dithering, tinta, foco, lente e degradação.	effects/*.py
Execução em background	Prévia e exportação sem bloquear a interface.	core/preview_render_controller.py, workers/*.py
Distribuição	Bundle ONEDIR, metadados do executável e instalador.	packaging/, build_*.ps1, installer/

Fluxo de dados

A carga normaliza a orientação EXIF e converte a imagem para RGBA. A sessão conserva o original em resolução completa e cria uma cópia reduzida para visualização. Cada mudança de controle gera um snapshot de PipelineSettings. O controlador de prévia limita o trabalho concorrente e descarta resultados antigos; a exportação usa um worker separado e processa a imagem original.

Estado e threads

O PreviewRenderController mantem um QThreadPool de um worker e substitui requisições pendentes pela mais recente. Sinais trazem o resultado ao thread da interface. O ExportWorker recebe imagem, settings e caminho de saída, reporta progresso e captura exceções como sinal de erro. A interface permanece dona dos widgets; os workers trabalham com copias de imagem e snapshots de settings.

3. Pipeline de imagem

A classe ImagePipeline implementa a sequência visual compartilhada. Os valores de progresso reportados sao aproximados e servem para acompanhar a exportação.

ORDEM	ETAPA	EFEITO
1	Tom	Contraste, esmagamento de pretos, queima de realces e gamma.
2	Paleta e dithering	Quantização em paletas RGB explicitas com algoritmos de dithering e escala.
3	Estrutura do dithering	Distorsao do padrão, perda de pontos e espalhamento.
4	Gravura	Reforço e direção de detalhes de borda com espessura configurável.
5	Consumo de preto	Expansão espacial de massas escuras e controle de detalhes absorvidos.
6	Tinta e papel	Sangramento, perda de tinta, grão de papel, speckles e desalinhamento de impressão.
7	Mapa de foco	Máscara geométrica, posição, largura, suavidade, rotação e desfoque.
8	Lente	Foco irregular, borramento da lente e difusão suave.
9	Degradação final	Grão de filme, poeira, riscos, desgaste de bordas, variação de exposição, vinheta e fade.

Paletas e cor

As paletas sao tabelas RGB explicitas com 2 a 6 tons. A luminância do pixel escolhe o índice de cor mais proximo, e o canal alfa original e preservado. A família Aged Ivory atual pode receber matiz de 0 a 359 graus e influência de 0 a 100; influência zero devolve as cores-base originais. O tingimento usa espaço Lab com cromaticidade deliberadamente moderada.

Escala espacial

A prévia usa uma imagem limitada a 1200 pixels no maior lado. Na exportação, spatial_scale ajusta parâmetros que dependem de distancia ou espessura (por exemplo, escala de dithering, espessura da gravura e alcance) para acompanhar a resolução real. O resultado não é ampliacao da miniatura: é reprocessamento da fonte original.

4. Recursos e fluxo de uso

Fluxo principal

PASSO	AÇÃO	RESULTADO
1	Importar ou arrastar PNG, JPG/JPEG, WebP ou BMP.	Imagem orientada via EXIF, aberta em RGBA e adicionada a sessão.
2	Selecionar a imagem na faixa da sessão.	A imagem torna-se ativa sem substituir o original.
3	Ajustar controles ou carregar um preset.	Nova prévia assíncrona; controles compoem um snapshot do pipeline.
4	Desfazer/refazer ou copiar/colar configurações.	Histórico limitado e transferivel entre imagens.
5	Exportar PNG.	Processamento em resolução completa com progresso e metadata recuperavel.
6	Reabrir o PNG exportado.	Quando metadata compativel existe, o usuário pode carregar as configurações embutidas.

Organizacao da interface

A janela apresenta barra de ações, lista de imagens e áreas de visualização original/resultado lado a lado. O zoom e o pan sao sincronizados; atalhos incluem + e - para zoom, Ctrl+Z e Ctrl+Shift+Z/Ctrl+Y para histórico, além de Ctrl+Shift+C/V para copiar/colar settings. O painel lateral agrupa preset, tom, paleta, dithering, gravura, consumo de preto, tinta, foco, lente e degradação.

Presets

Os presets de fábrica ficam em presets/factory_presets.json e sao mesclados com valores default para permitir definicoes parciais. Os presets pessoais ficam em data/user_presets.json, sao validados contra o schema atual e aceitam salvar, atualizar, renomear e excluir. A migração de configurações legadas de Aged Ivory converte nomes antigos para a paleta parametrizada atual.

5. Persistência e configurações

A pasta de dados gravável fica ao lado do executável em modo empacotado, ou na raiz do projeto durante execução pelo código-fonte. O caminho e validado por uma tentativa real de escrita antes do uso. Em falha, AppDataError e exibido ao usuário.

ARQUIVO	CONTEÚDO	REPOSITÓRIO
data/settings.ini	Preferências QSettings da janela e da interface; migração unica de QSettings legado quando ainda não existe arquivo portátil.	Ignorado pelo Git; dados locais.
data/user_presets.json	Presets pessoais versionados no formato de dados 1.	Ignorado pelo Git; dados locais.
presets/factory_presets.json	Overrides de presets de fábrica sobre os valores default.	Fonte versionada.
presets/presets.json	Catalogo de presets distribuido junto ao código.	Fonte versionada.
PNG / iTXt	Snapshot serializado de PipelineSettings sob a chave ruinprint.settings.	Dentro do arquivo exportado pelo usuário.

Modelo de configuração

PipelineSettings agrega dataclasses imutáveis para tone, palette, dither, dither_destruction, engraving, black_consumption, ink, focus, lens e degradation. A conversao para dicionario e compartilhada entre presets, histórico, metadata e controles. A validação exige a estrutura completa, tipos compativeis e limites para quantidade de tons, escala e controles de matiz/influência.

Recomendacao operacional

Não versionar data/settings.ini nem data/user_presets.json. Sao dados de cada usuário e podem incluir preferências ou presets pessoais. A distribuição inicia somente com dados da aplicação e cria a pasta data quando necessario.

6. Metadata PNG e recuperação

Cada PNG exportado recebe uma entrada iTXt chamada ruinprint.settings. O payload contém identificação da aplicação, tipo, versão do schema e um snapshot completo das configurações. Não inclui a imagem original nem uma cópia da imagem fonte.

Envelope lógico

CAMPO	EXEMPLO / REGRA
app	RuinPrint
type	processing-settings
schema_version	1
settings	Objeto completo compativel com PipelineSettings.

Na importacao, apenas PNGs sao consultados. JSON malformado, schema diferente, payload de outro app ou configurações invalidas sao ignorados com segurança. A recuperação e oferecida como ação explícita para a imagem ativa; ela altera o estado do editor, não os pixels do PNG carregado. A migração de variantes legadas de Aged Ivory aplica hue/influence equivalentes antes da validação.

Compatibilidade e evolução

O schema versionado evita assumir que futuros formatos mantem os mesmos campos. Ao alterar as dataclasses, os mantenedores devem decidir como migrar snapshots anteriores, preservar nomes de campos quando possivel e acrescentar validação antes de ativar a recuperação. Exportar novamente produz o schema atualmente declarado.

7. Build, instalador e distribuição

Build Windows

ARQUIVO	PAPEL
build_windows.ps1	Confere a virtualenv local, limpa apenas build e dist/RuinPrint e executa PyInstaller com packaging/RuinPrint.spec. Cópia settings.ini/user_presets.json existentes para a pasta data da build.
packaging/RuinPrint.spec	Define executável RuinPrint ONEDIR, carrega factory_presets.json, icon opcional e console=False.
build_installer.ps1	Localiza ISCC.exe (Inno Setup 7 ou 6) e compila o script do instalador para release/.
installer/RuinPrint.iss	Instalador por usuário, atalho do Menu Iniciar e opcional de desktop; preserva dados existentes e não os remove na desinstalação.
build_release.ps1	Encadeia build portátil e compilação do instalador.
prepare_release.ps1	Monta pasta de distribuição com instalador, versão portátil e LEIA-ME.txt.

Estrutura de uma build portátil

RuinPrint/ RuinPrint.exe _internal/ data/ settings.ini user_presets.json

Configuração de release

A versão e declarada em core/app_info.py e duplicada em installer/RuinPrint.iss e build_installer.ps1. Uma nova release deve manter estes valores sincronizados, gerar a build em Windows, verificar os arquivos de distribuição e testar instalação/execução em ambiente separado do checkout de desenvolvimento. Outputs de build e executáveis não fazem parte do repositório-fonte.

8. Estrutura de pastas

Mapa resumido do repositório observado. Os diretórios de dados locais e de build sao gerados ou específicos por usuário e ficam fora do controle de versão.

RuinPrint/ app.py requirements.txt README.md .gitignore assets/ ruinprint-mark.png core/ app_info.py app_paths.py edit_history.py exporter.py image_document.py image_loader.py pipeline.py preset_manager.py processing_settings.py preview_render_controller.py effects/ tone.py, palette.py, dither.py, dither_modifiers.py engraving.py, black_consumption.py, ink.py focus/lens/degradation modules ui/ main_window.py, controls.py, image_view.py, theme.py workers/ preview_worker.py, export_worker.py presets/ factory_presets.json, presets.json packaging/ RuinPrint.spec, version_info.txt installer/ RuinPrint.iss build_windows.ps1 build_installer.ps1 build_release.ps1 prepare_release.ps1 docs/ RuinPrint_Documentacao_Tecnica_PT-BR.pdf screenshots/

Convenções

- core concentra contratos e coordenacao, não os controles visuais por efeito.

- effects implementa transformações com parâmetros explicitos.

- workers contém adaptadores Qt para processamento em background.

- presets inclui dados de fábrica versionados; dados pessoais ficam em data/.

- docs/screenshots e a área reservada para capturas verdadeiras adicionadas posteriormente.

9. Decisoes tecnicas

DECISÃO	MOTIVO E EFEITO
Imagem original preservada	Edicao não destrutiva; controles sempre recomputam a partir da fonte.
Snapshot imutavel de settings	Prévias e exportação compartilham uma configuração coerente mesmo quando os controles mudam durante uma tarefa.
Prévia assíncrona mais recente	Evita bloquear a UI e evita fila de prévias obsoletas durante arraste de controles.
RGBA de ponta a ponta	Preserva transparencias de entrada durante a quantização da paleta.
Metadata dentro de PNG	Permite transportar ajustes com o resultado sem arquivo sidecar separado.
Diretorio data junto ao app	Favorece modo portátil e instalação autocontida; exige permissao de escrita na pasta da aplicação.
PyInstaller ONEDIR	Distribuição com dependencias em pasta interna; mais simples para diagnosticar/copiar que one-file, mas a pasta precisa permanecer completa.

Limites conhecidos

- A interface e documentada como português brasileiro; não ha fluxo de localizacao observado.

- As receitas de build e instalador estão voltadas ao Windows. O repositório não demonstra pipeline de release para macOS ou Linux.

- A prévia e limitada a 1200 px; o custo de cada efeito em imagem muito grande depende do tamanho e dos parâmetros escolhidos.

- O schema de metadata e versionado, mas o código observado define atualmente a versão 1.

- Não foi encontrado suite de testes automatizados versionada na árvore revisada.

- A pasta de trabalho tinha mudanças de código fora de commit durante a revisão. Elas precisam de revisão antes de representar uma release pública.

Licença

Nenhum arquivo LICENSE foi encontrado. A documentação não escolhe termos legais pelo autor. Antes de convidar contribuições ou permitir reutilização, o proprietário deve selecionar uma licença e adiciona-la ao repositório.

10. Próximos passos

PRIORIDADE	AÇÃO
Apresentação	Substituir as caixas reservadas por capturas reais do aplicativo e incluir uma imagem de antes/depois autorizada.
Revisão	Revisar cada diff local existente e decidir quais mudanças de funcionalidade pertencem ao primeiro snapshot público.
Licenciamento	Escolher uma licença consciente sobre direitos autorais, redistribuição e uso comercial; publicar arquivo LICENSE.
Release	Conferir sincronizacao de versão, gerar build limpa no Windows e testar instalador e modo portátil fora da pasta de desenvolvimento.
Qualidade	Definir uma estratégia de testes para schema de settings, alpha, pipeline determinístico e metadata PNG.
Distribuição	Criar release versionada e anexar pacotes binários sem armazenar executáveis no Git.

Preparacao profissional para GitHub

O README apresenta objetivo, recursos, tecnologias, diagrama e instruções de desenvolvimento. O .gitignore exclui ambientes virtuais, dados locais e builds. O PDF fornece contexto tecnico. A primeira publicação deve conter apenas arquivos revisados; não enviar presets pessoais, settings, imagens de clientes, executáveis ou outros arquivos locais por engano.

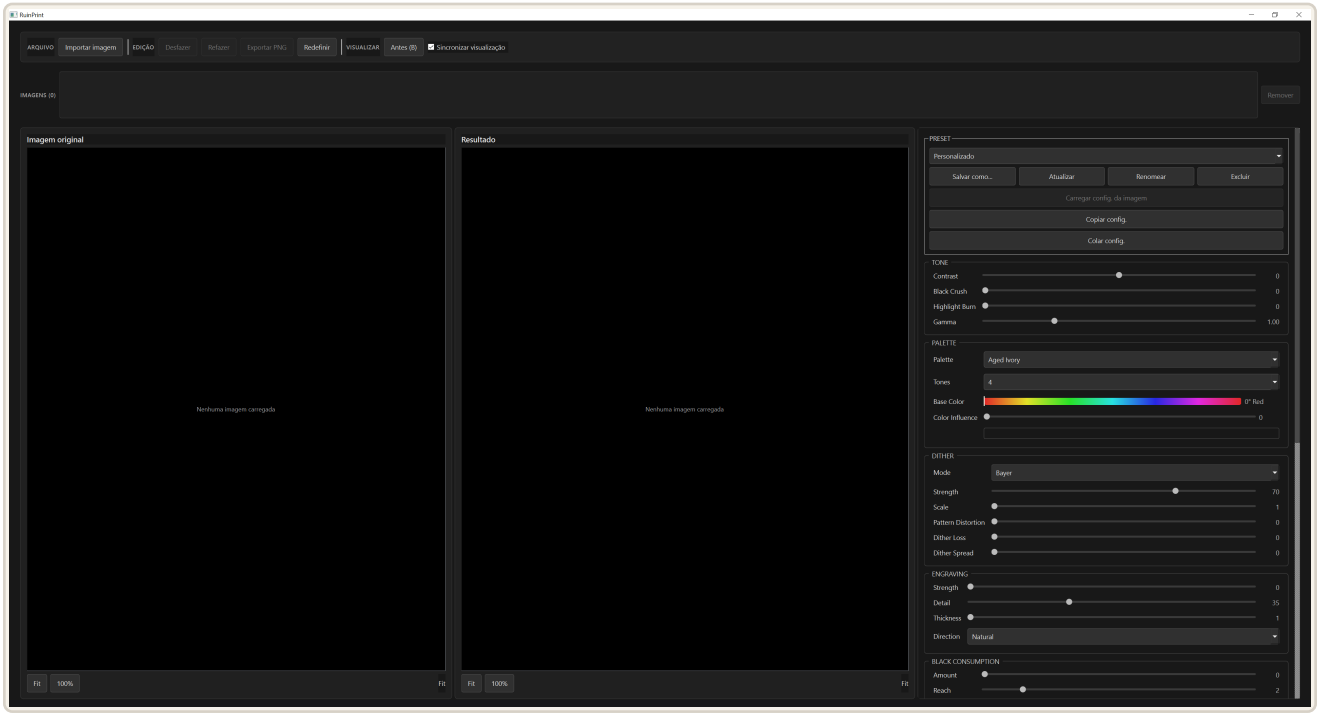
Comandos para o primeiro envio

1. Revise o estado e os diffs locais. 2. Escolha quais mudanças de aplicação integrar. 3. Crie no GitHub um repositório público chamado RuinPrint, se esse for o destino aprovado. 4. Adicione origin com a URL do novo repositório. 5. Adicione seletivamente a documentação e os arquivos de aplicação aprovados, crie um commit e envie master. 6. Confirme no GitHub o README, o PDF e os arquivos esperados.

Não ha origin configurado neste checkout. O README inclui um exemplo de comandos, mas o repositório remoto precisa existir antes de git push.

12. Interface e sessão de imagens

A janela reúne navegação, comparação e controles em um único espaço de edição.



Visão geral da interface

A captura mostra a barra de ações, a lista de imagens, as duas áreas de visualização e o painel de ajustes.



Sessão com várias imagens

A faixa superior permite selecionar a imagem ativa e remover itens da sessão.

13. Comparação e resultado

As vistas Original e Resultado ajudam a avaliar como os parâmetros mudam a imagem.

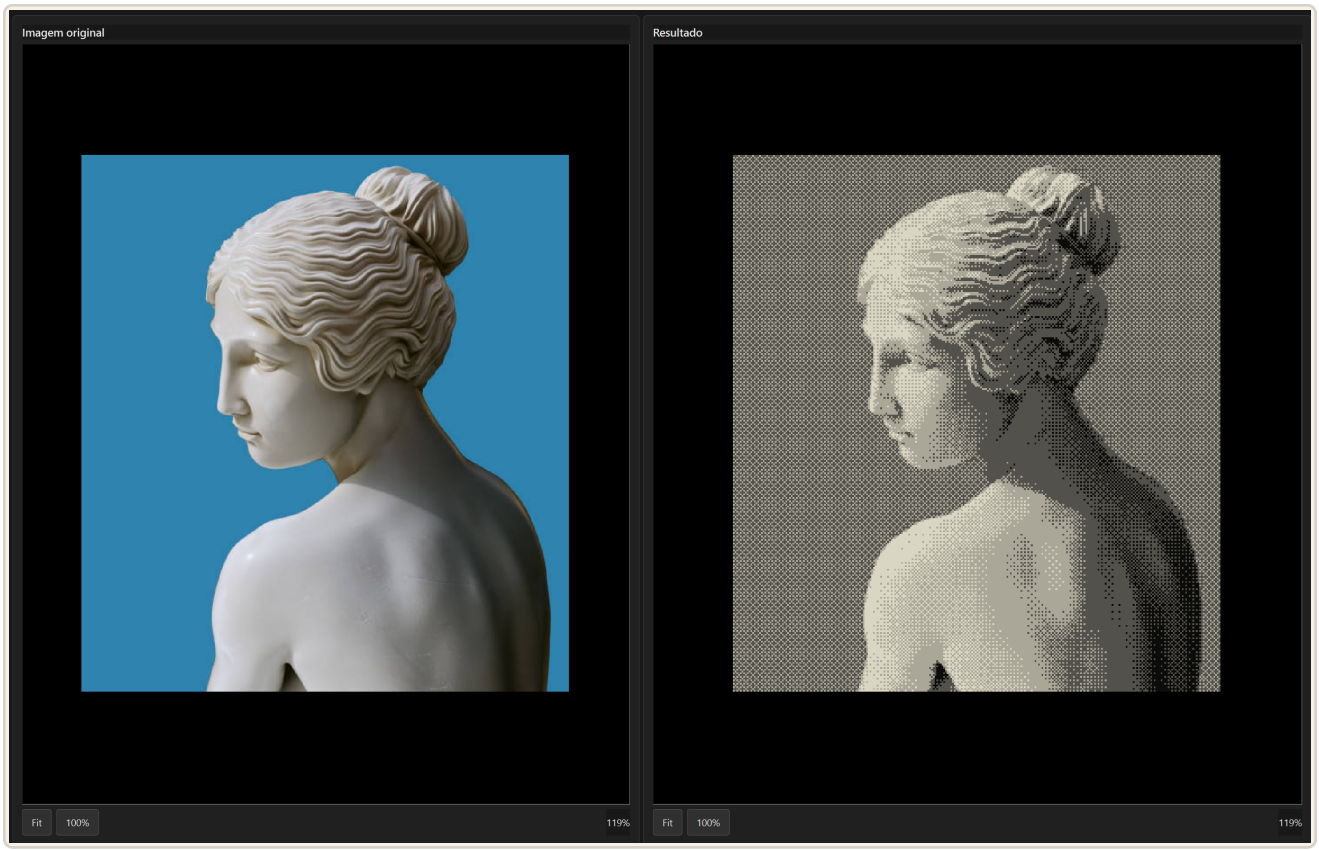
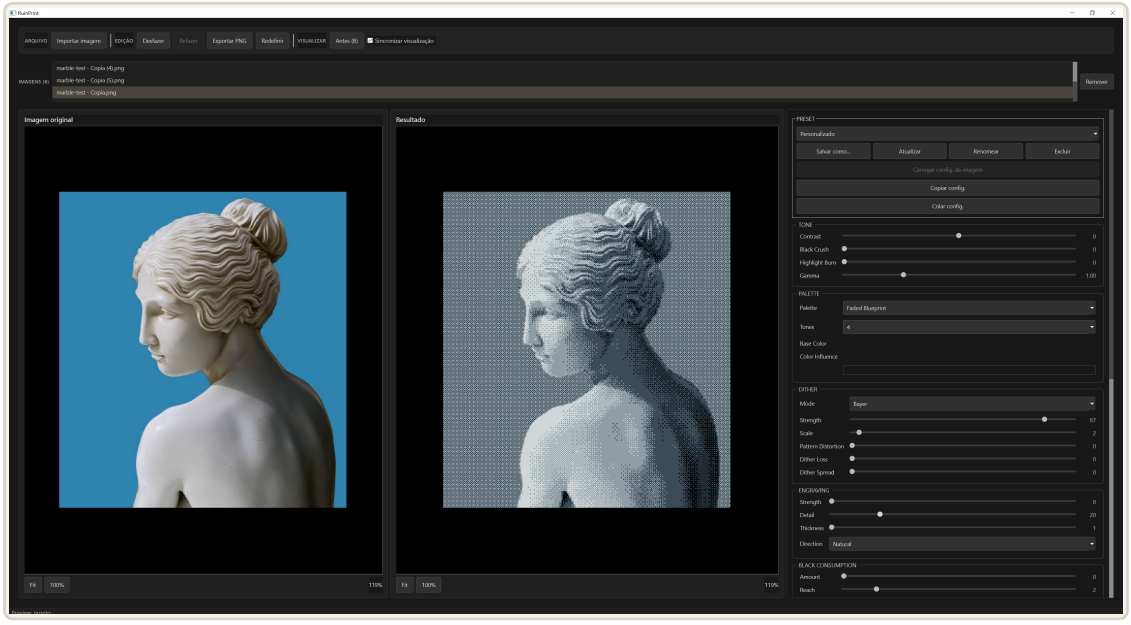


Imagem original e resultado

Exemplo de comparação lado a lado com dithering e a paleta Aged Ivory.

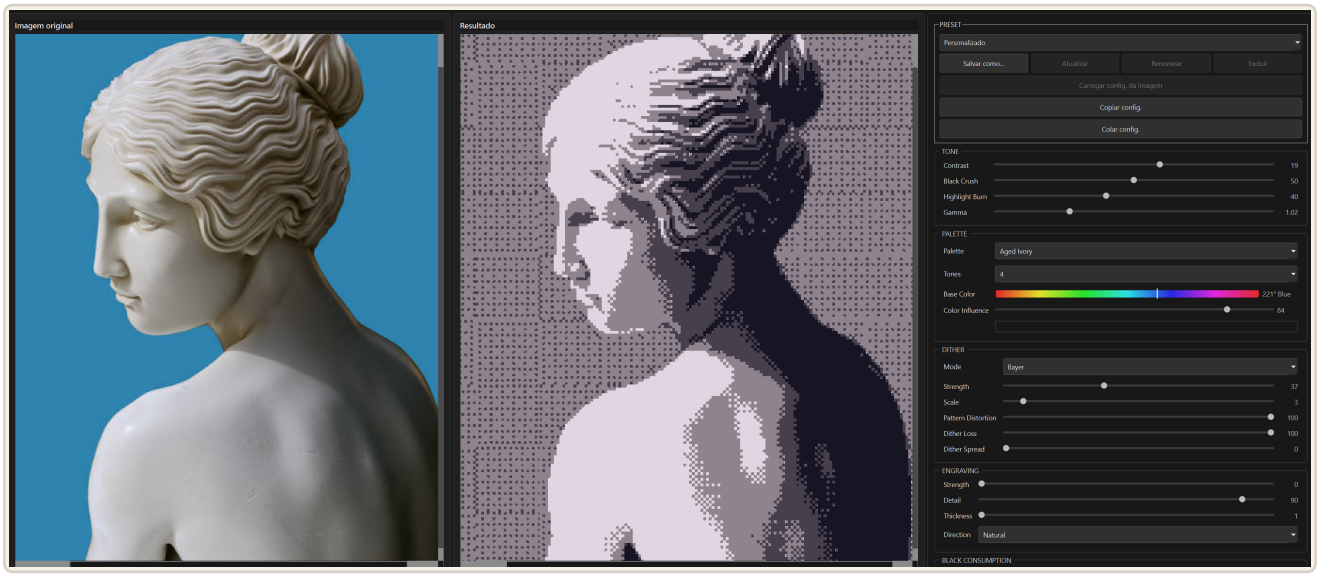


Edição em andamento

A captura inclui a sessão com várias imagens e os controles usados para compor um resultado.

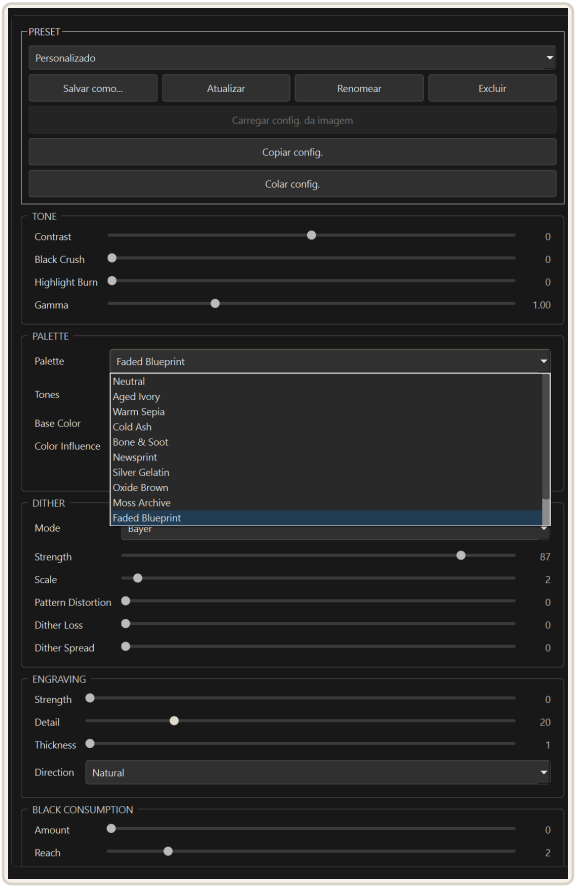
14. Paletas e cor-base

O controle de matiz permite escolher uma base cromática para a família Aged Ivory.



Aged Ivory com base azul

A captura mostra matiz em 221° (Blue) e influência de cor em 84%.

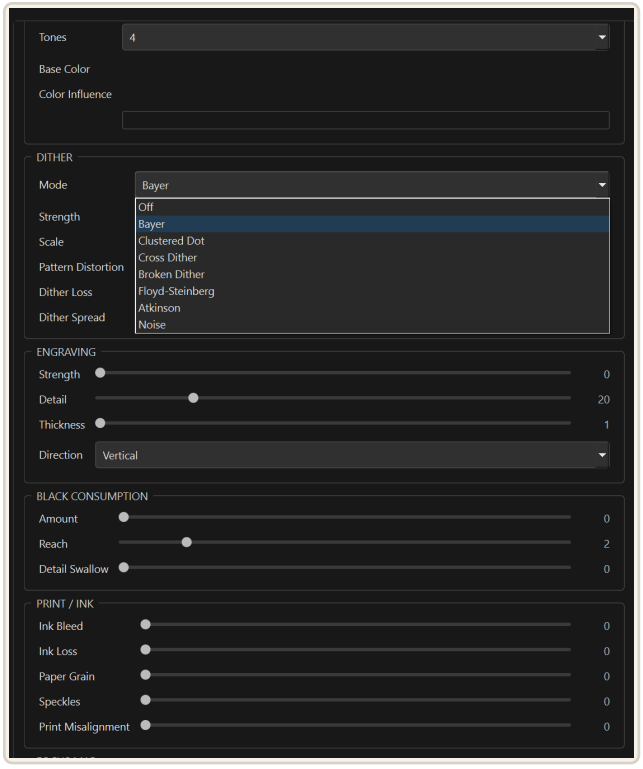


Paletas disponíveis

Lista de paletas visíveis na interface. A seleção mostrada é Faded Blueprint.

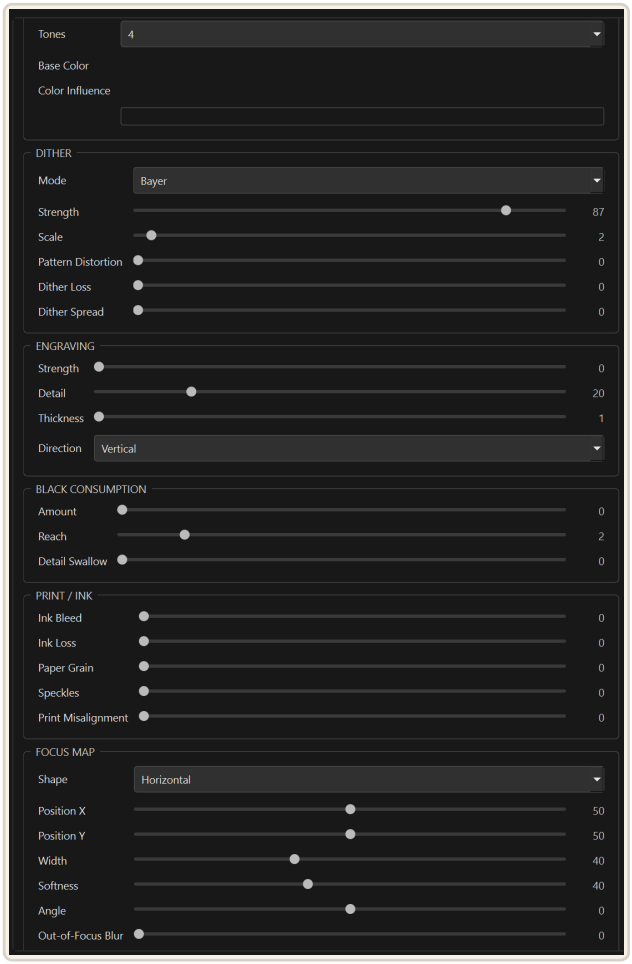
15. Dithering e efeitos

Os grupos de controles mantêm os ajustes organizados por etapa do pipeline.



Modos de dithering

Algoritmos de dithering, escala, intensidade e modificadores do padrão.



Controles do pipeline

Gravura, consumo de preto, tinta e mapa de foco em grupos próprios.